Dynamic Programming Study Guide

What is Dynamic Programming?

Dynamic Programming (DP) is an algorithmic technique that solves complex problems by breaking them down into simpler subproblems. It stores the results of subproblems to avoid redundant calculations, making the solution more efficient.

Key Characteristics:

- **Optimal Substructure**: The problem can be broken down into smaller subproblems
- **Overlapping Subproblems**: The same subproblems are solved multiple times
- **Memoization**: Store results to avoid recalculation

Dynamic Programming Approaches

1. Top-Down Approach (Memoization)

Uses recursion with a cache to store computed results.

```java
// Example: Fibonacci with memoization
public class Fibonacci {
    private static Map<Integer, Long> memo = new HashMap<>();

    public static long fibonacci(int n) {
        if (n <= 1) return n;

        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        long result = fibonacci(n - 1) + fibonacci(n - 2);
        memo.put(n, result);
        return result;
    }
}
```

2. Bottom-Up Approach (Tabulation)

Builds the solution iteratively from the smallest subproblems.

```java
// Example: Fibonacci with tabulation
public static long fibonacci(int n) {
    if (n <= 1) return n;

    long[] dp = new long[n + 1];
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}
```

Practice Session on Dynamic Programming (Knapsack Type Questions)

1. 0/1 Knapsack Problem

Problem: Given weights and values of items, find the maximum value that can be obtained with a weight limit.

```java
public class Knapsack01 {
    public static int knapsack(int[] weights, int[] values, int capacity) {
        int n = weights.length;
        int[][] dp = new int[n + 1][capacity + 1];

        // Build table dp[][] in bottom-up manner
        for (int i = 0; i <= n; i++) {
            for (int w = 0; w <= capacity; w++) {
                if (i == 0 || w == 0) {
                    dp[i][w] = 0;
                } else if (weights[i - 1] <= w) {
                    dp[i][w] = Math.max(
                        values[i - 1] + dp[i - 1][w - weights[i - 1]],
                        dp[i - 1][w]
                    );
                } else {
                    dp[i][w] = dp[i - 1][w];
                }
            }
        }

        return dp[n][capacity];
    }

    public static void main(String[] args) {
        int[] values = {60, 100, 120};
        int[] weights = {10, 20, 30};
        int capacity = 50;

        System.out.println("Maximum value: " + knapsack(weights, values, capacity));
    }
}
```

2. Unbounded Knapsack Problem

Problem: Items can be used unlimited times.

```java
public class UnboundedKnapsack {
    public static int unboundedKnapsack(int[] weights, int[] values, int capacity) {
        int[] dp = new int[capacity + 1];

        for (int i = 1; i <= capacity; i++) {
            for (int j = 0; j < weights.length; j++) {
                if (weights[j] <= i) {
                    dp[i] = Math.max(dp[i], dp[i - weights[j]] + values[j]);
                }
            }
        }

        return dp[capacity];
    }

    public static void main(String[] args) {
        int[] weights = {1, 3, 4, 5};
        int[] values = {1, 4, 5, 7};
        int capacity = 7;

        System.out.println("Maximum value: " + unboundedKnapsack(weights, values, capacity));
    }
}
```

3. Subset Sum Problem

Problem: Check if there exists a subset with a given sum.

```java
public class SubsetSum {
    public static boolean isSubsetSum(int[] arr, int sum) {
        int n = arr.length;
        boolean[][] dp = new boolean[n + 1][sum + 1];

        // If sum is 0, answer is true
        for (int i = 0; i <= n; i++) {
            dp[i][0] = true;
        }

        // If sum is not 0 and set is empty, answer is false
        for (int i = 1; i <= sum; i++) {
            dp[0][i] = false;
        }

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= sum; j++) {
                if (j < arr[i - 1]) {
                    dp[i][j] = dp[i - 1][j];
                } else {
                    dp[i][j] = dp[i - 1][j] || dp[i - 1][j - arr[i - 1]];
                }
            }
        }

        return dp[n][sum];
    }

    public static void main(String[] args) {
        int[] arr = {3, 34, 4, 12, 5, 2};
        int sum = 9;

        System.out.println("Subset exists: " + isSubsetSum(arr, sum));
    }
}
```

4. Coin Change Problem

Problem: Find minimum number of coins needed to make a given amount.

```java
public class CoinChange {
    public static int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;

        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (coin <= i) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
            }
        }

        return dp[amount] > amount ? -1 : dp[amount];
    }

    public static void main(String[] args) {
        int[] coins = {1, 3, 4};
        int amount = 6;

        System.out.println("Minimum coins needed: " + coinChange(coins, amount));
    }
}
```

5. Partition Equal Subset Sum

Problem: Check if array can be partitioned into two equal sum subsets.

```java
public class PartitionEqualSubset {
    public static boolean canPartition(int[] nums) {
        int sum = 0;
        for (int num : nums) {
            sum += num;
        }

        if (sum % 2 != 0) return false;

        int target = sum / 2;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;

        for (int num : nums) {
            for (int j = target; j >= num; j--) {
                dp[j] = dp[j] || dp[j - num];
            }
        }

        return dp[target];
    }

    public static void main(String[] args) {
        int[] nums = {1, 5, 11, 5};
        System.out.println("Can partition: " + canPartition(nums));
    }
}
```

Common DP Patterns in Knapsack Problems

Pattern Recognition

- **0/1 Knapsack**: Each item can be taken at most once
- **Unbounded Knapsack**: Items can be taken unlimited times
- **Subset Sum**: Find if subset exists with given sum
- **Coin Change**: Find minimum ways to make change
- **Partition**: Divide array into equal parts

Time and Space Complexity

- **0/1 Knapsack**: Time O(n*W), Space O(n*W)
- **Unbounded Knapsack**: Time O(n*W), Space O(W)
- **Subset Sum**: Time O(n*sum), Space O(n*sum)

Quiz Questions

Question 1: What is the main advantage of using Dynamic Programming over naive recursion? a) It uses less memory b) It avoids redundant calculations by storing results c) It is easier to implement d) It works faster for all problems

Answer: b) It avoids redundant calculations by storing results

Question 2: In the 0/1 Knapsack problem, what does dp[i][w] represent? a) The weight of item i b) The value of item i c) Maximum value achievable using first i items with weight limit w d) The total number of items

Answer: c) Maximum value achievable using first i items with weight limit w

Question 3: What is the difference between 0/1 Knapsack and Unbounded Knapsack? a) 0/1 uses recursion, Unbounded uses iteration b) 0/1 allows each item once, Unbounded allows unlimited use c) 0/1 is faster than Unbounded d) There is no difference

Answer: b) 0/1 allows each item once, Unbounded allows unlimited use

Question 4: In the Subset Sum problem, what does dp[i][j] represent? a) The sum of first i elements b) Whether subset sum j is possible using first i elements c) The maximum sum possible d) The number of subsets

Answer: b) Whether subset sum j is possible using first i elements

Question 5: Which approach builds the solution from smaller subproblems iteratively? a) Top-down approach b) Memoization c) Bottom-up approach d) Recursive approach

Answer: c) Bottom-up approach

Practice Coding Questions

Question 1: Target Sum Problem Given an array of integers and a target sum, find the number of ways to assign + or - signs to make the sum equal to target.

Question 2: Minimum Subset Sum Difference Given an array, partition it into two subsets such that the absolute difference between their sums is minimized. Return the minimum difference.

Question 3: Count of Subset Sum Given an array and a sum, find the total number of subsets with the given sum.

Question 4: Coin Change II Given coins of different denominations and a total amount, find the number of combinations that make up that amount.

Question 5: Perfect Squares Given a positive integer n, find the least number of perfect square numbers that sum to n.